

Traiter les questions suivantes:**Exercice 1: (11 pts)**

On considère la classe *Product* suivante:

```
public class Product {
    // Produit en français
    private String name;           // nom en français
    private int quantity;          // quantité en français
    private String category;       // catégorie en français

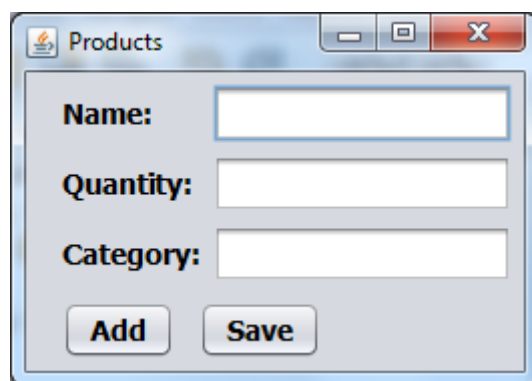
    public Product(String name, int quantity, String category) {
        this.name = name;
        this.quantity = quantity;
        this.category = category;
    }

    public String getName() {
        return name;
    }

    public int getQuantity() {
        return quantity;
    }

    public String getCategory() {
        return category;
    }
}
```

On considère l'interface graphique suivante permettant de saisir les données d'un produit dans des champs de texte. Le bouton *Add* permet de créer un nouvel objet *Product* et de l'ajouter à un *ArrayList*. Le bouton *Save* permet de sauvegarder les produits de l'*ArrayList* dans un fichier. Les données du fichier sont chargées au démarrage du programme à l'aide de la méthode *load* ci-dessous.



Le code correspondant à cette interface graphique est comme suit:

```
public class ProductFrame extends Frame {
    private TextField name = new TextField(10);
    private TextField quantity = new TextField(10);
    private TextField category = new TextField(10);
    private Label lName = new Label("Name:");
    private Label lQuantity = new Label("Quantity:");
    private Label lCategory = new Label("Category:");
    private Button add = new Button("Add");
    private Button save = new Button("Save");
    private ArrayList<Product> productsList =
        new ArrayList<Product>();

    public ProductFrame() {
        // à compléter ... (1)
        this.pack();
        load();
        // à compléter ... (2)
    }

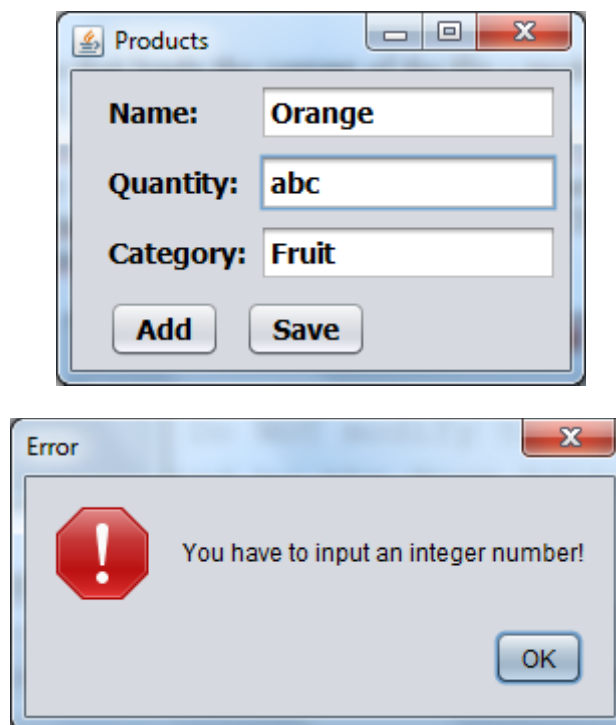
    private void addPerformed() {
        // à compléter ... (3)
    }
}
```

```
private void savePerformed() {
    // à compléter ... (4)
}

public void save2Performed() {
    // à compléter ... (5)
}

private void load() {
    // à compléter ... (6)
}
}
```

- Donner en (1) le code nécessaire afin d'obtenir l'interface graphique ci-dessus. (Vous pouvez utiliser les gestionnaires *BorderLayout*, *FlowLayout*, et *GridLayout*).
- Donner en (2) le code qui permet d'ajouter un écouteur d'action (*ActionListener*) au bouton *Add* et qui exécute la méthode *addPerformed*.
- Donner en (3) le code permettant de saisir les champs de texte, de créer un objet *Product*, et de l'ajouter à l'objet *productsList*. Les champs doivent être vides après l'ajout.
- Donner en (4) le code permettant d'ouvrir le fichier « product.txt » et d'y écrire les informations des produits de la *productsList*. Chaque ligne doit contenir les informations d'un seul produit comme suit :
Orange,10,Fruit
- Donner en (5) le code permettant de sérialiser (persister) les objets *Product* de la *productsList* dans un fichier « data.bin ». Quel changement doit-on effectuer à la classe *Product* pour la persistance?
- Donner en (6) le code permettant de charger les informations contenues dans le fichier « product.txt », de créer des objets *Product* (un par ligne) et de les ajouter à la *productsList*.
- La méthode *parseInt* soulève une exception du type *NumberFormatException* si la chaîne (String) passée en paramètre ne peut pas être convertie en un nombre entier. Changer le code de la méthode *addPerformed()* afin de supporter cette exception. L'appel de cette exception affichera un *JOptionPane* comme suit:



Exercice 2: (5pts)

- Ecrire le code permettant de dessiner et de remplir un cercle à l'intérieur d'un carré. Les dimensions du carré ainsi que sa couleur sont entrées dans le code html.
- Modifier le code précédent pour afficher un message d'erreur si les paramètres du carré sont supérieurs aux dimensions de l'applet.

Exercice 3: (4pts)

Supposons qu'on a une base de données appelée « bookDB » avec un username « hello » et un password « welcome ».

Cette base de données contient la table « Book » qui est composée de deux attributs: « code » et « title » avec les tuples (enregistrements) suivants:

Code	Title
JV	Java
DB	Database
OS	Operating System
DS	Data Structures

- Ouvrir une connexion à la base de données, lire les données de cette table et les placer dans un objet de type *HashMap*.
- Utiliser un itérateur sur la *HashMap* déjà créée et ensuite afficher chaque tuple sur une ligne.